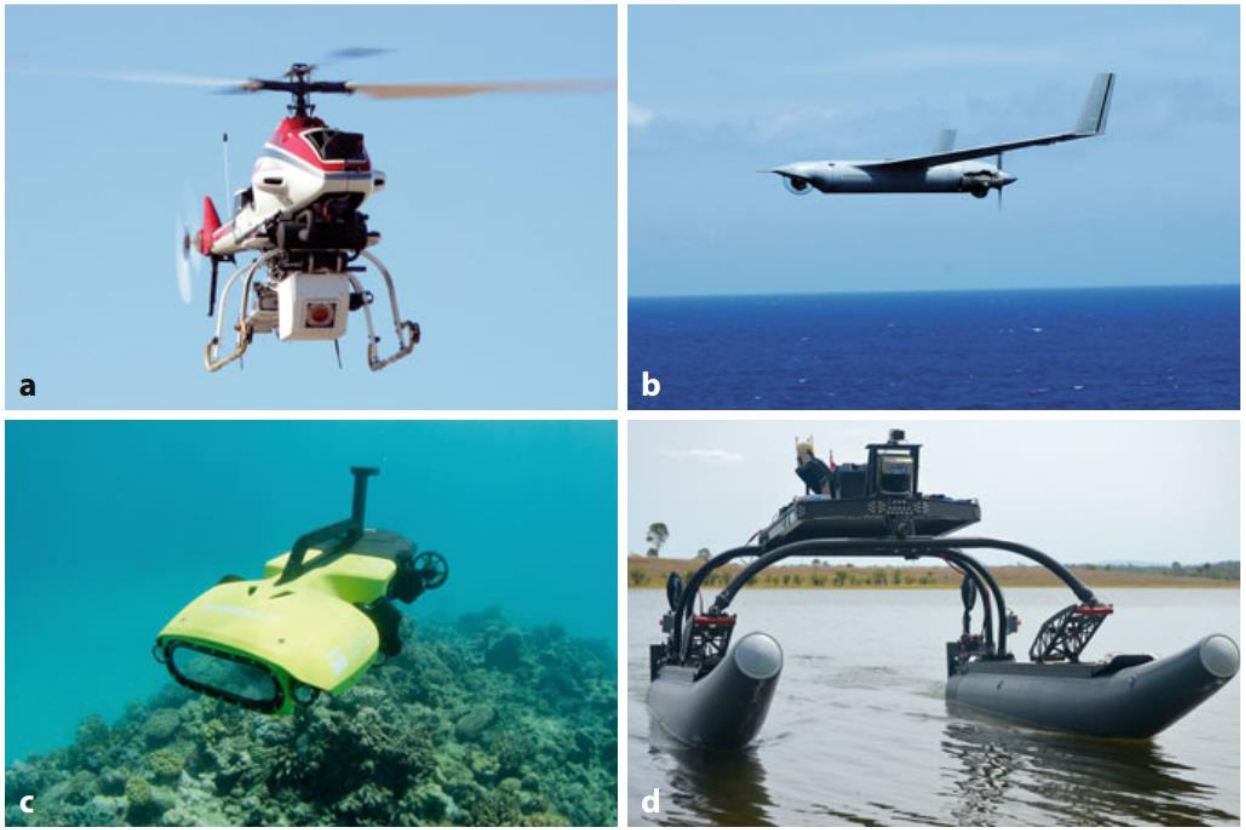
Chapter 3.
Mobile Robots & Control



- This chapter discusses how a robot platform moves, that is, how its pose changes with time as a function of its control inputs.





Contents



3.1 Mobile Robots



3.2 Control

DRAFT

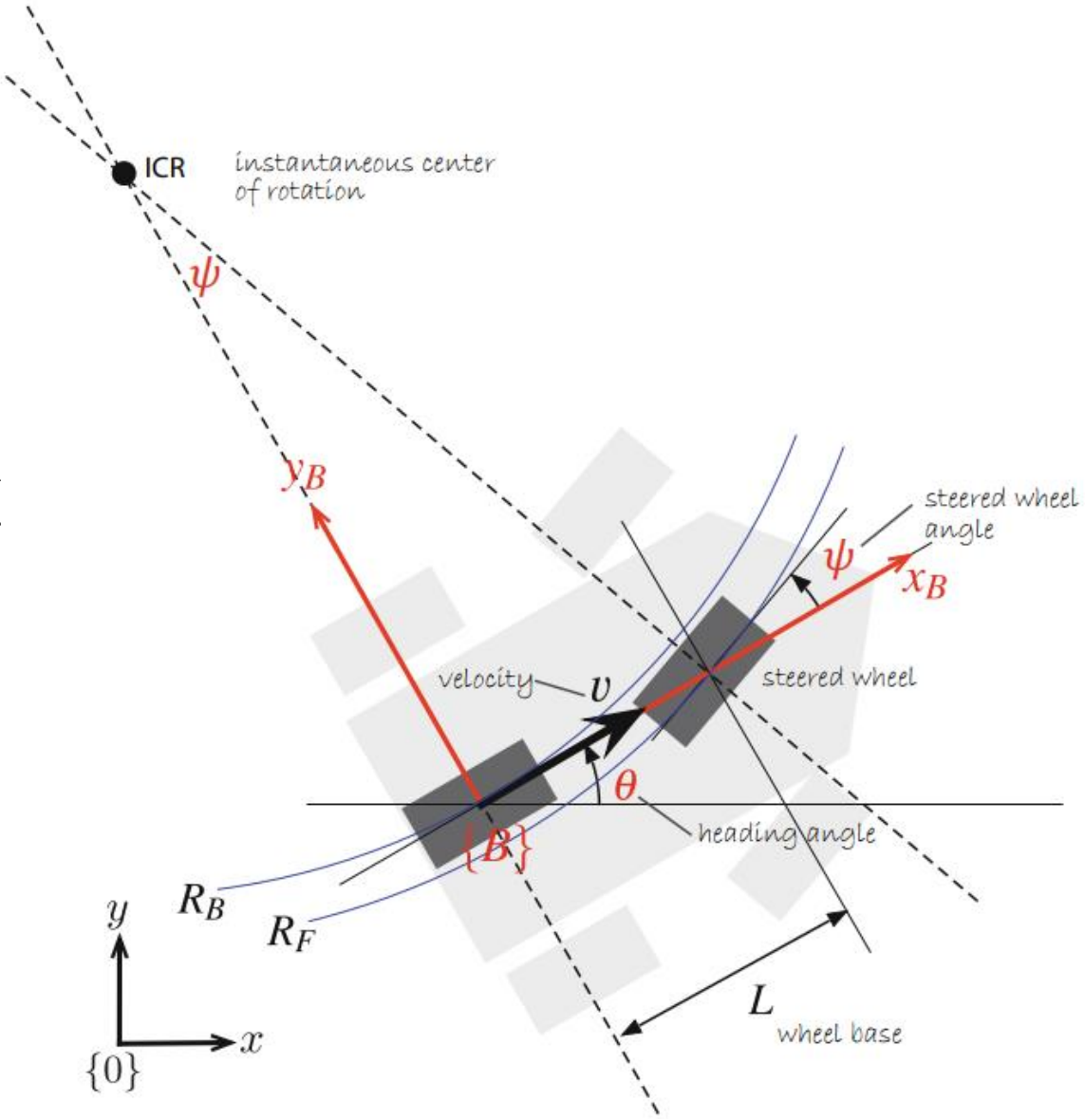
3.1 Wheeled Mobile Robots

- In this section we will create a model for a car-like vehicle, and develop controllers that can drive the car to a point, along a line, follow an arbitrary path, and finally, drive to a specific pose.

DRAFT

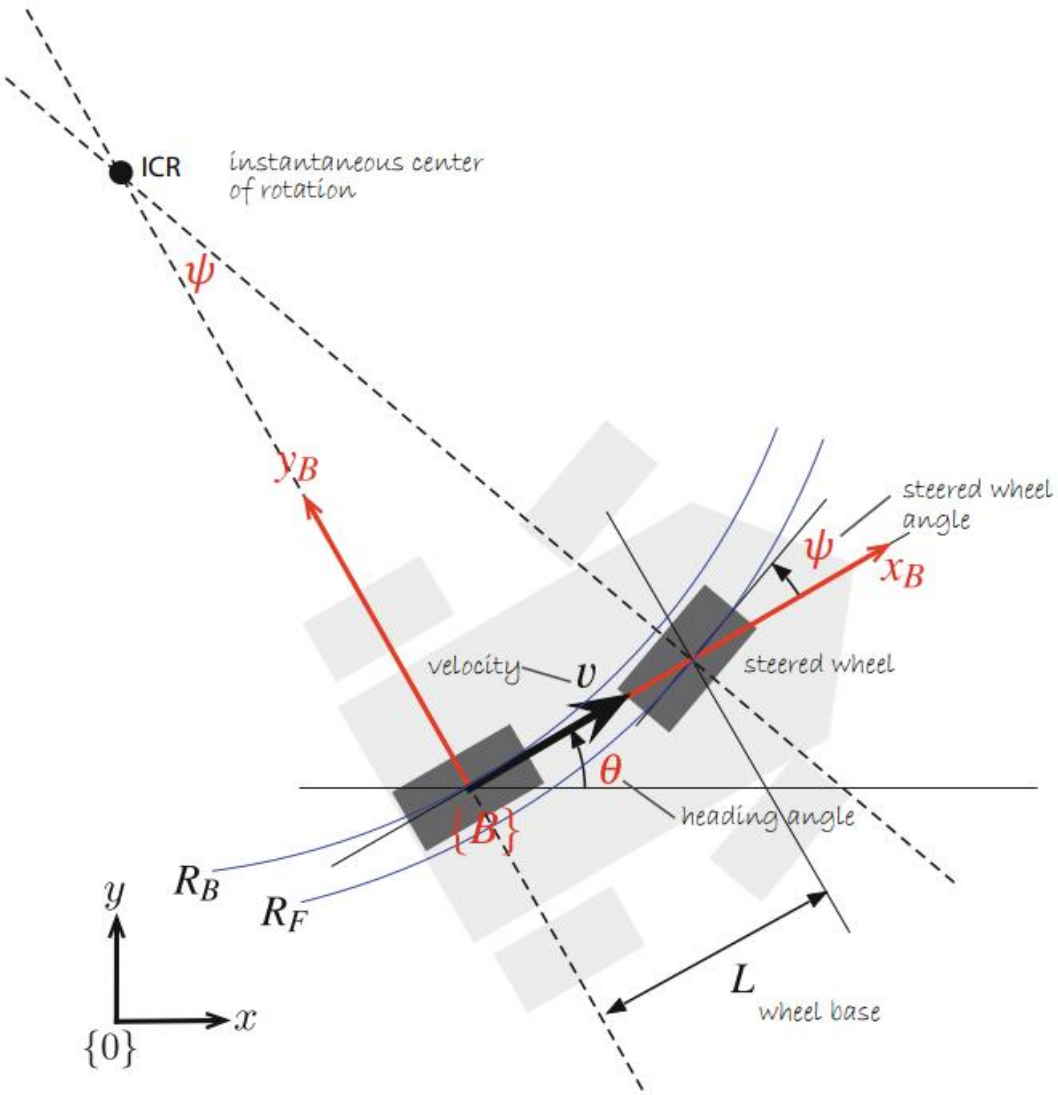
3.1.1 Mobile robot modeling

- A commonly used model for the low-speed behavior of a four-wheeled car-like vehicle is the kinematic bicycle model.
- The bicycle has a rear wheel fixed to the body and the plane of the front wheel rotates about the vertical axis to steer the vehicle.
- The pose of the vehicle is represented by its body coordinate frame {B}, which has its x-axis in the vehicle's forward direction and its origin at the center of the rear axle.



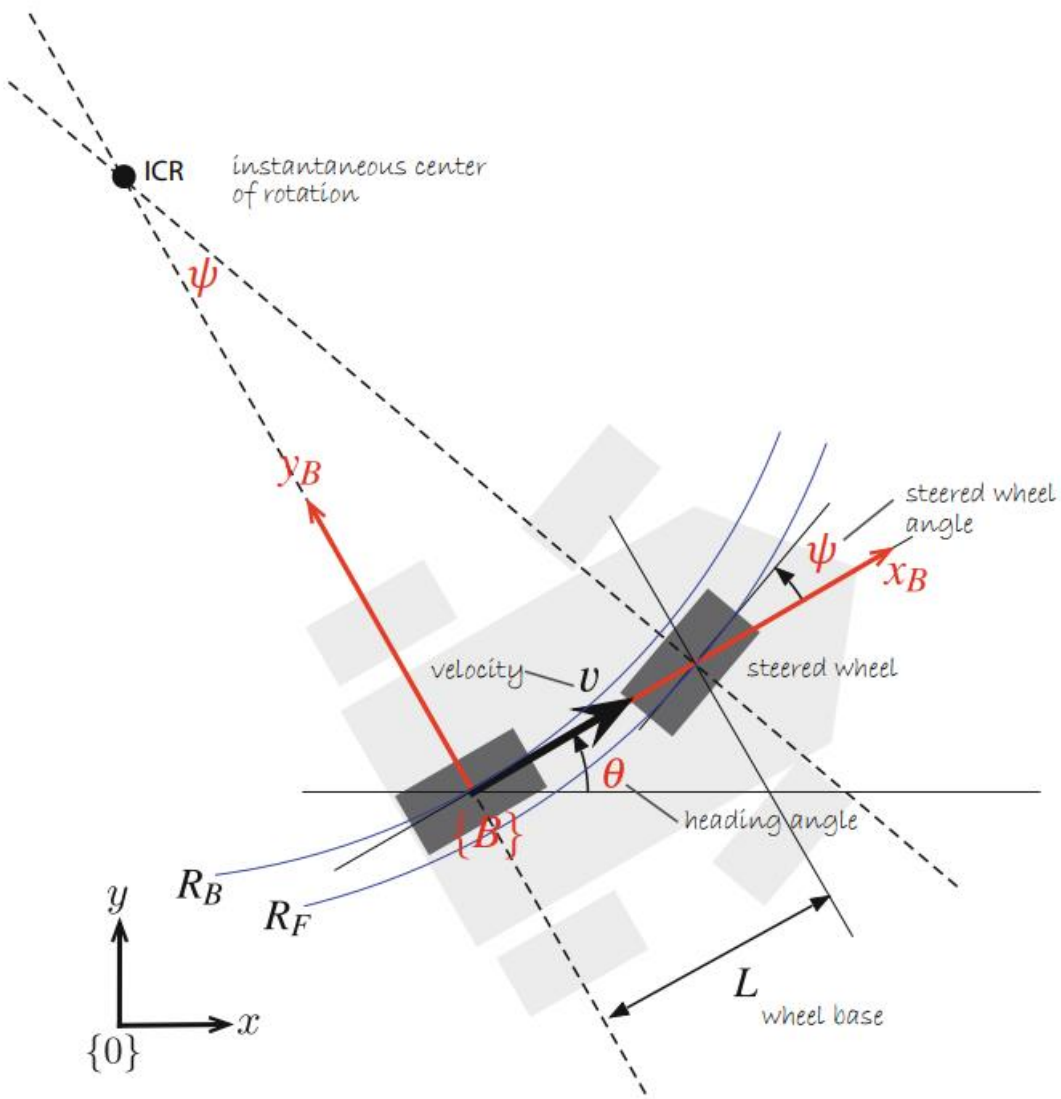
DRAFT

- Bicycle model of a car-like vehicle.
- The 4-wheeled car is shown in light gray, and the equivalent two-wheeled bicycle model in dark gray.
- The vehicle's body frame is shown in red, and the world coordinate frame in black.
- The steered wheel angle is with respect to the body frame, and the velocity of the back wheel, in the body's x-direction, is v.



DRAFT

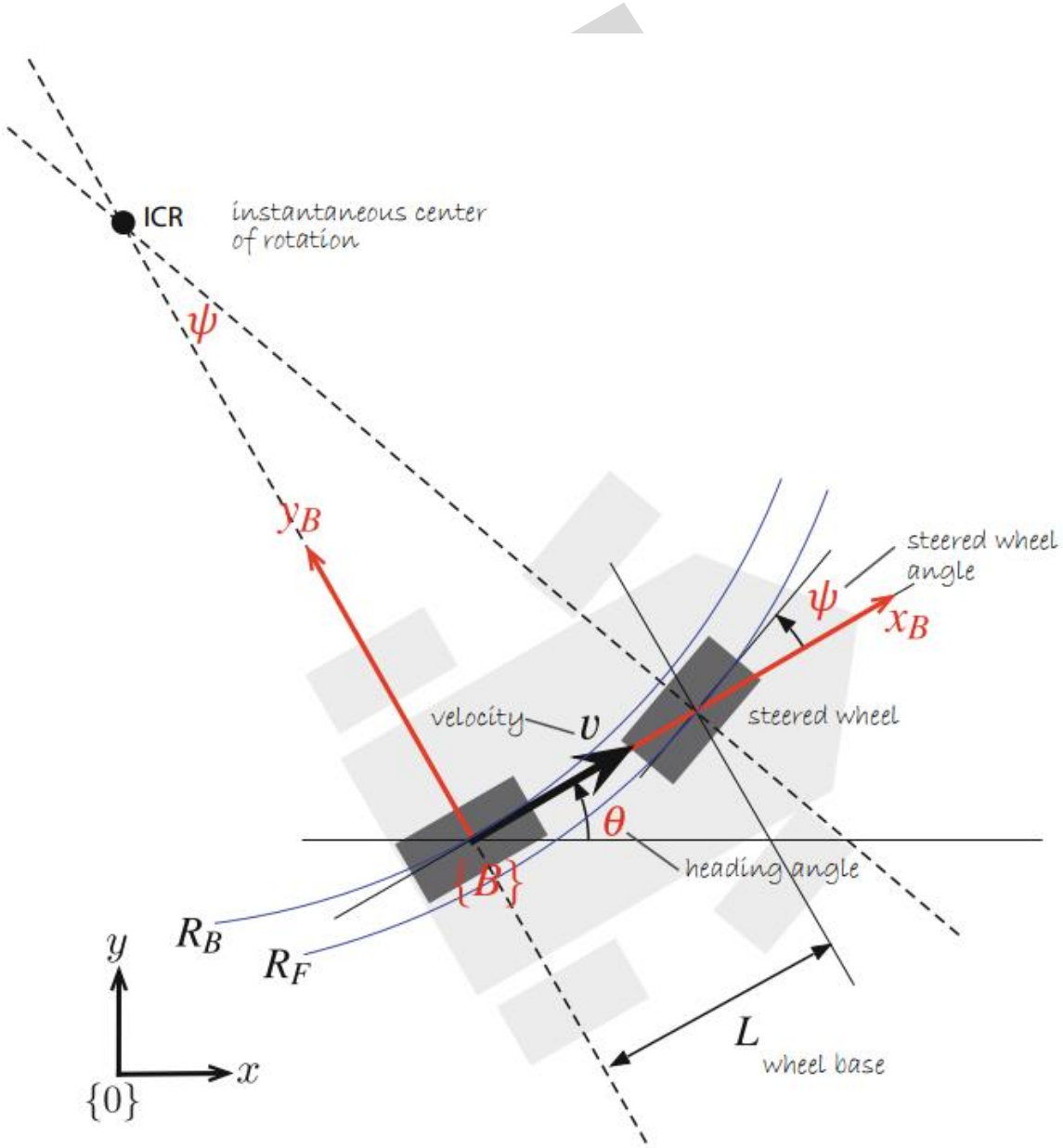
- The two wheel axes are extended as dashed lines and intersect at the instantaneous center of rotation (ICR).
- The back and front wheels follow circular arcs around the ICR of radius R_B and R_F respectively.



DRAFT

- The orientation of its x-axis with respect to the world frame is the vehicle's heading angle.
- The configuration of the vehicle is represented by the generalized coordinates $\boldsymbol{q} = (x, y, \theta) \in \mathcal{C}$ where $\mathcal{C} \subset \mathbb{R}^2 \times \mathbf{S}^1$ is the configuration space.
- We assume that the velocity of each wheel is in the plane of the wheel, and that the wheels roll without skidding or slipping sideways

$$^B\boldsymbol{v} = (v, 0)$$



- The reference point of the vehicle, the origin of frame {B}, thus follows a circular path and its angular velocity is

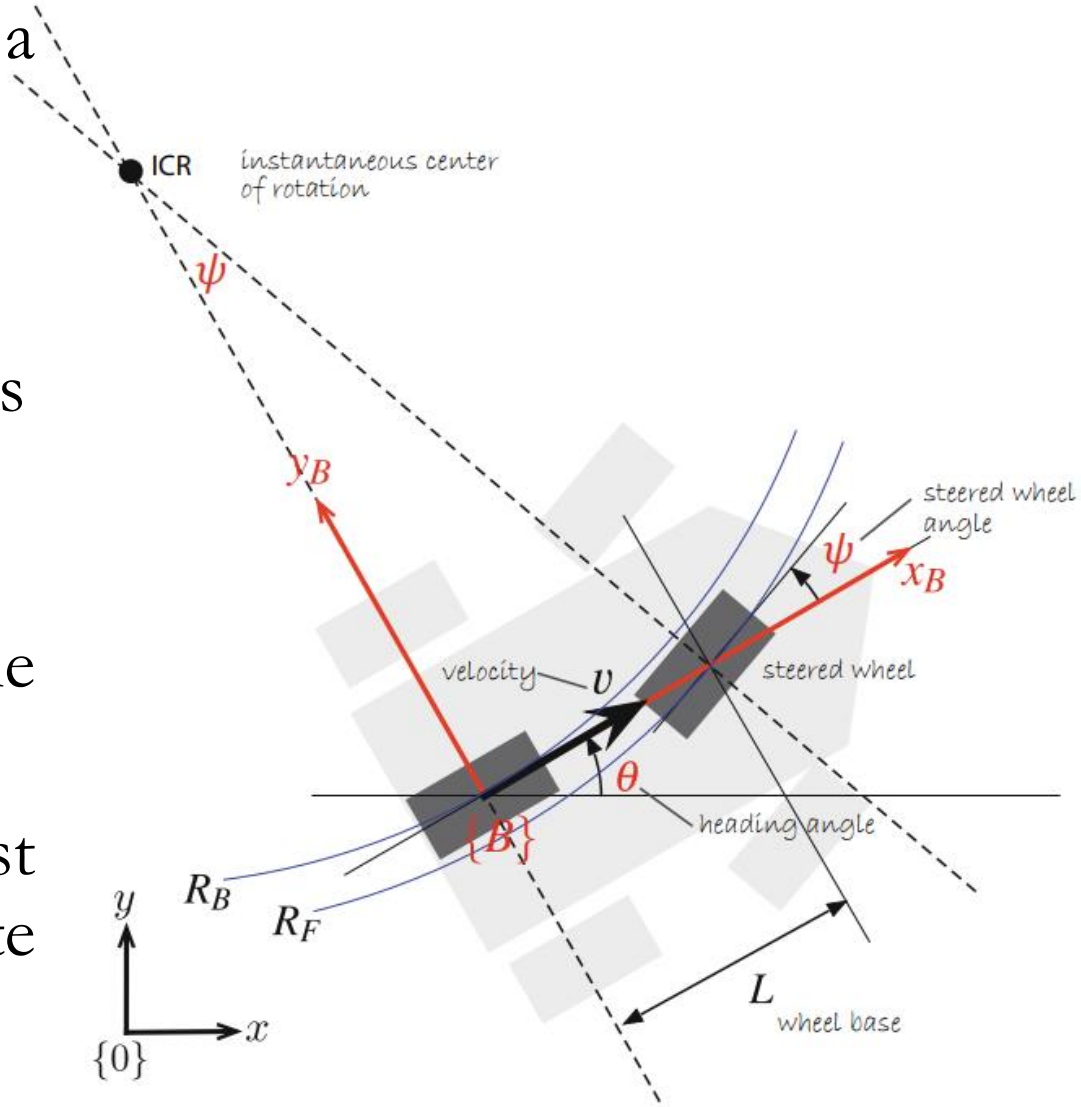
$$\dot{\theta} = \frac{v}{R_B}$$

and by simple geometry the turning radius is

$$R_{\mathrm{B}} = \frac{L}{\tan \psi}$$

where L is the length of the vehicle or the wheel base.

- Since $R_F > R_B$, the front wheel must follow a longer path and therefore rotate more quickly than the back wheel.



DRAFT

Excurse: Vehicle Coordinate System

> The coordinate system that we will use, and a common one for vehicles of all sorts, is that the x-axis is forward (longitudinal motion), the y-axis is to the left side (lateral motion), which implies that the z-axis is upward. For aerospace and underwater applications, the x-axis is forward, and the z-axis is often downward.

DRAFT

Excurse: Ackermann Steering

Rudolph Ackermann (1764–1834) was a German inventor born in Schneeberg, Saxony. For financial reasons, he was unable to attend university and became a saddler like his father. He worked as a saddler and coachbuilder, and in 1795 established a print shop and drawing school in London. He published a popular magazine *The Repository of Arts, Literature, Commerce, Manufactures, Fashion and Politics* that included an eclectic mix of articles on water pumps, gaslighting, and lithographic presses, along with fashion plates and furniture designs. He manufactured paper for landscape and miniature painters, patented a method for waterproofing cloth and paper and built a factory in Chelsea to produce it. He is buried in Kensal Green Cemetery, London.

In 1818, Ackermann took out British patent 4212, on behalf of the German inventor George Lankensperger, for a steering mechanism for coaches that ensures that the steered wheels move on circles with a common center. The same scheme was proposed and tested by Erasmus Darwin (grandfather of Charles Darwin) in the 1760s. Subsequent refinement by the Frenchman Charles Jeantaud led to the mechanism used in cars to this day, which is known as Ackermann steering.



- The velocity of the robot in the world frame is described by

$$\dot{x} = v\cos\theta$$
$$\dot{y} = v\sin\theta$$
$$\dot{\theta} = \frac{v}{L}\tan\psi$$

which are the equations of motion, or the motion model.

  - The rate of change of heading $\dot{\theta}$ is referred to as turn rate, heading rate, or yaw rate and can be measured by a gyroscope.
  - When $v = 0$, then $\dot{\theta} = 0$: not possible to change the vehicle's orientation when it is not moving.
  - When the steering angle $\psi = \pi/2$, the vehicle cannot move forward (undefined region).

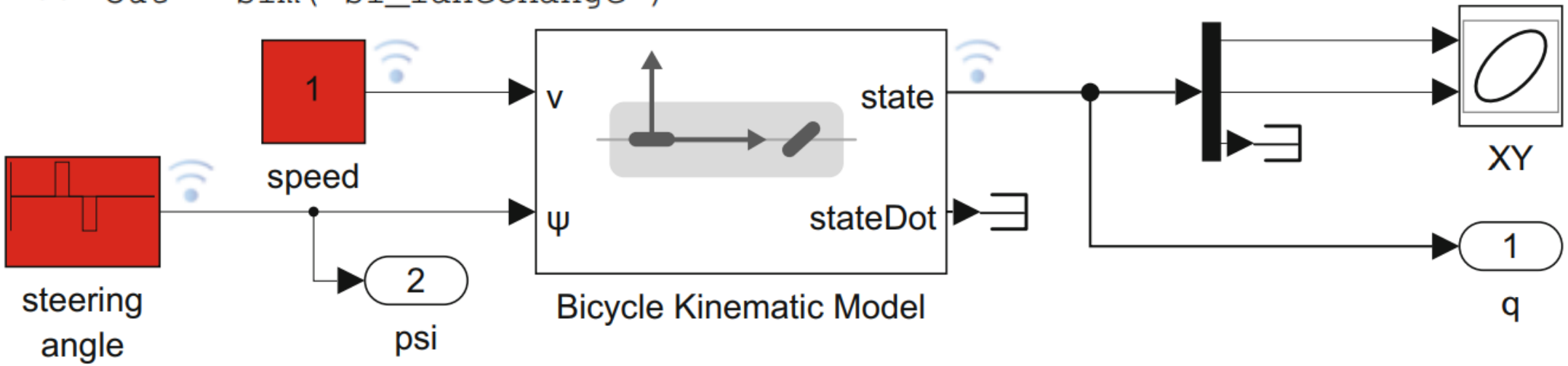
- Nonholonomic constraint: $\dot{y}\cos\theta - \dot{x}\sin\theta \equiv 0$

DRAFT

3.1.2 Lane changing

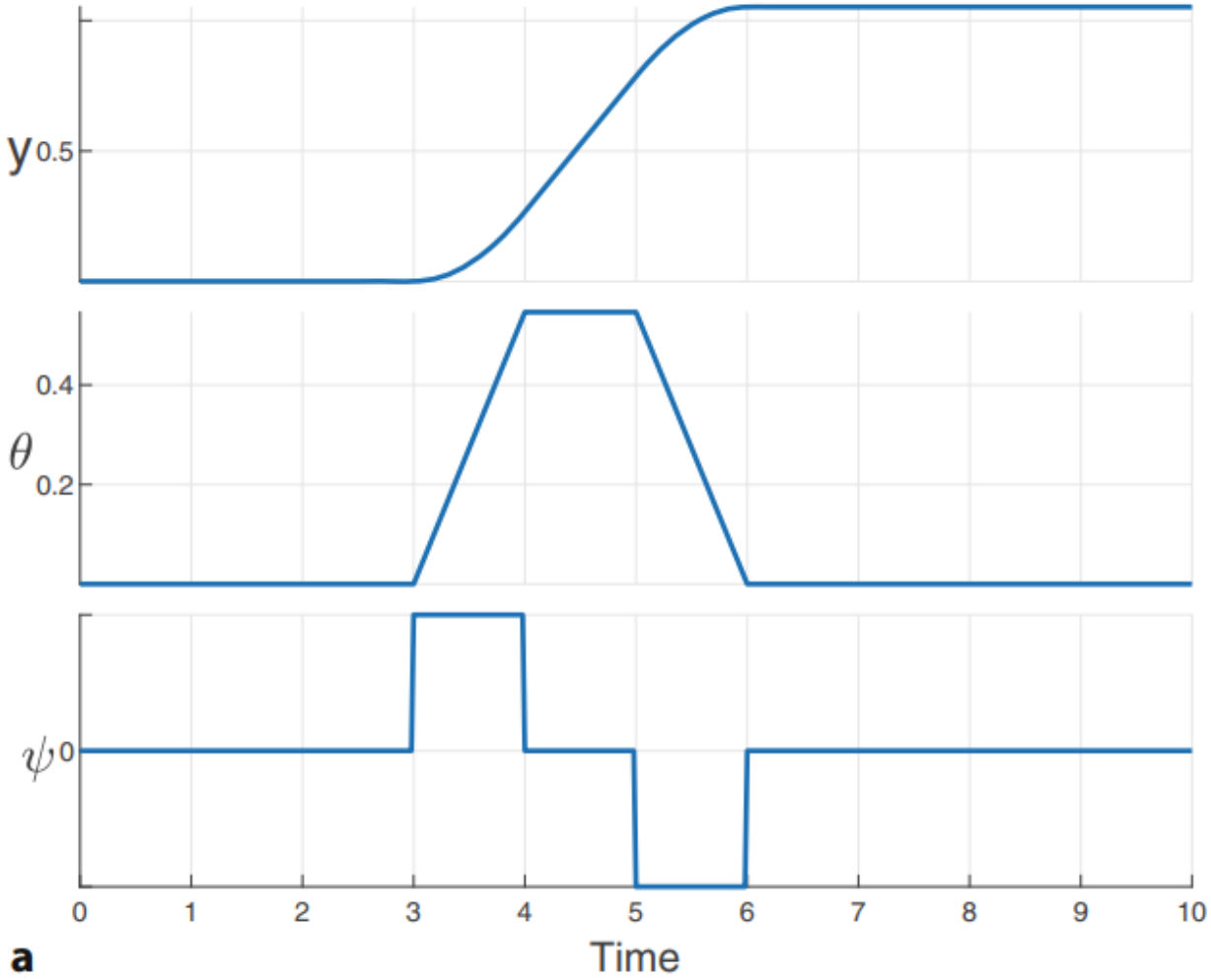
- We can explore the behavior of the vehicle for simple steering input using the Simulink® model.

```
>> sl_lanechange
>> out = sim("sl_lanechange")
```

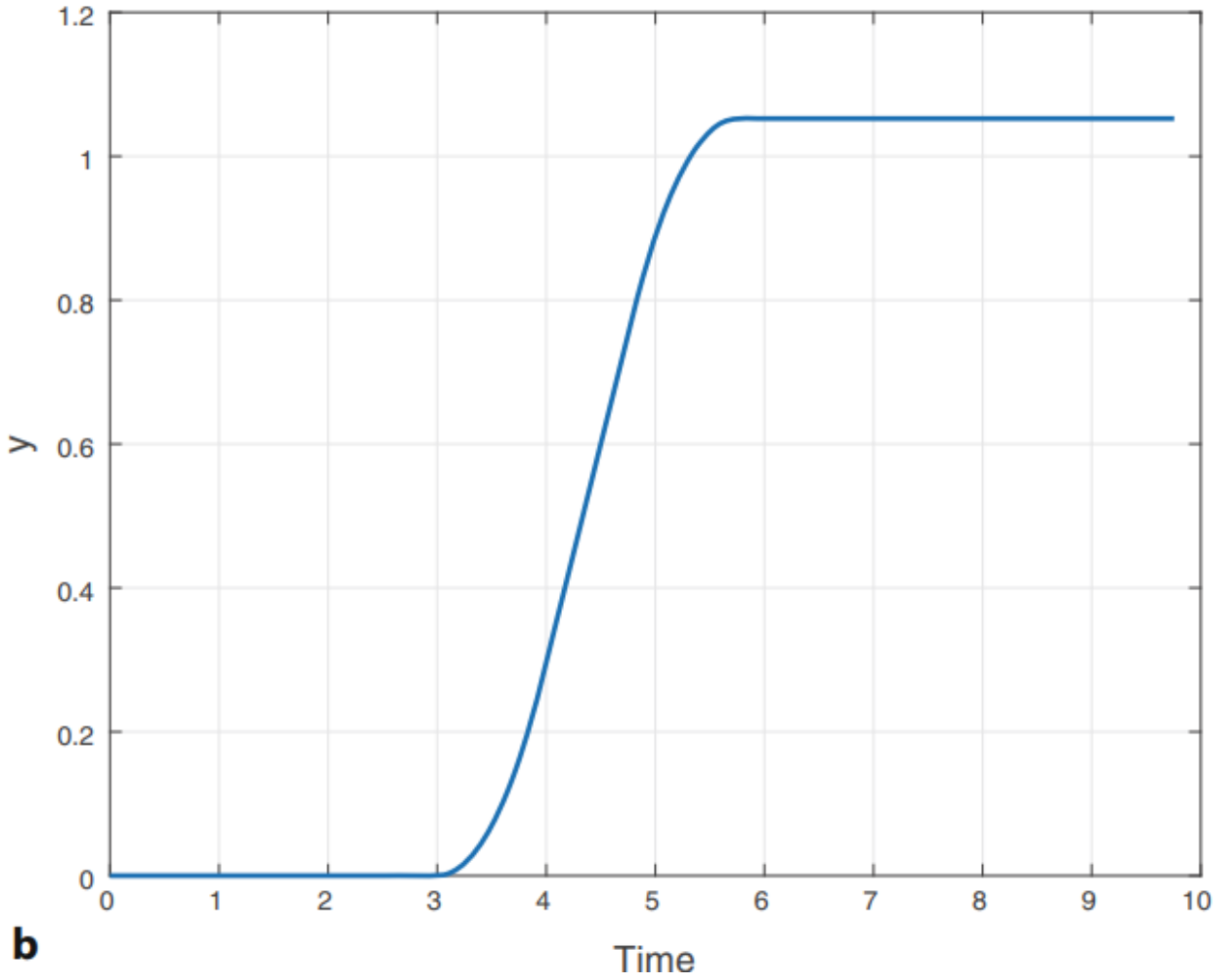



```
out =
  Simulink.SimulationOutput:
                logsout: [1x1 Simulink.SimulationData.Dataset]
                      t: [504x1 double]
                      y: [504x4 double]
     SimulationMetadata: [1x1 Simulink.SimulationMetadata]
           ErrorMessage: [0x0 char]
```

```
>> t = out.get("t");
>> q = out.get("y");
>> stackedplot(t,q, ...
>>   DisplayLabels=["x","y","theta","psi"], ...
>>   GridVisible=1,XLabel="Time")
>> plot(q(:,1),q(:,2))
```



a



b

Excurse: Running Simulink Models

To run the Simulink model called `model`, we first load it

```
>> model
```

and a new window is created that displays the model. The simulation can be started by pressing the `Run` button on the toolbar of the model's window. The model can also be run directly from the MATLAB® command line

```
>> sim("model")
```

Many RVC Toolbox models create additional figures to display robot animations or graphs as they run.

All models in this chapter have the simulation data export option set to create a `SimulationOutput` object. All the signals feeding into output ports are concatenated, in port number order, to form a row vector and these are stacked to form a matrix `y` with one row per timestep. The corresponding time values form a vector `t`. These variables are packaged in a `Simulink.SimulationOutput` object which is written to the workspace variable `out` or returned if the simulation is invoked from MATLAB

```
>> r = sim("model")
```

Displaying `r` or `out` lists the variables that it contains, and their value is obtained using the `find` method, for example

```
>> t = r.find("t");
```

Besides the inputs and outputs, you can also log arbitrary signals in the model. Simply left-click a signal line in the model and select `"Log Signal"` in the toolstrip. By default, data from this logging will be stored in `out.logsout` as a `Dataset` object .

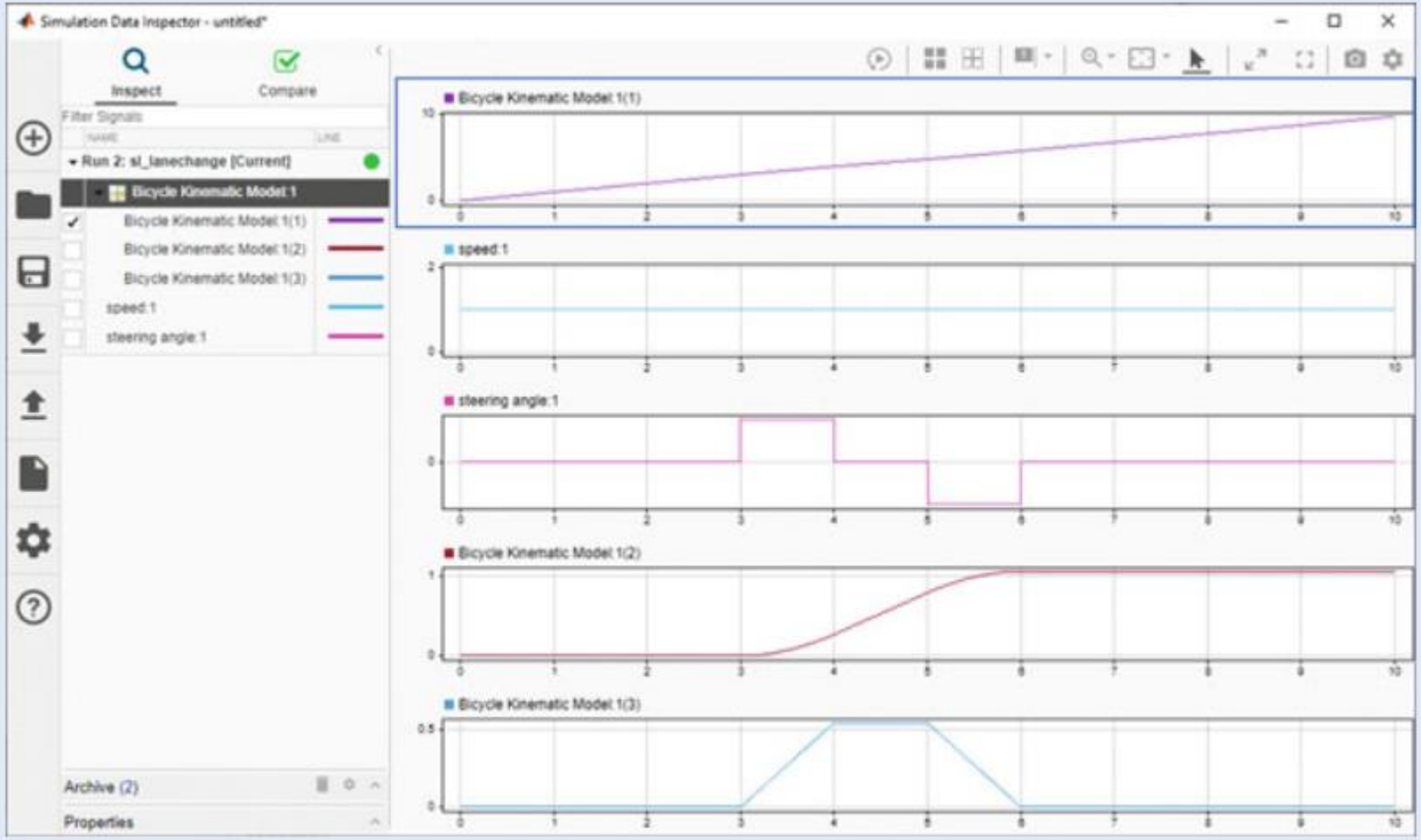


The advantage of the `"Log Signal"` workflow is that it is easy to visualize the simulation output with the Simulation Data Inspector. You can analyze a single simulation run and easily compare the outcomes of multiple runs by calling `plot` on the simulation output object

```
>> r.plot;
```

An example of the Simulation Data Inspector visualization after simulating the `sl_lanechange` model is shown below.

Excurse: Running Simulink Models (cont’)



3.1.3 Driving to a Point

- Consider the problem of moving toward a goal point (x*, y*) on the xy-plane.
- We will control the robot's velocity to be proportional to its distance from the goal.

$$v^* = K_v \sqrt{(x^* - x)^2 + (y^* - y)^2}$$

and to steer toward the goal, which is at the vehicle-relative angle in the world frame of

$$\theta^* = \tan^{-1} \frac{y^* - y}{x^* - x}$$

We use the operator $\ominus$ since θ^*, θ are angles, not real numbers, and the result is always in the interval $[-\pi, \pi)$.

- A simple proportional controller

$$\psi = K_h(\theta^* \ominus \theta), \quad K_h > 0$$

is sufficient – it turns the steered wheel toward the target.

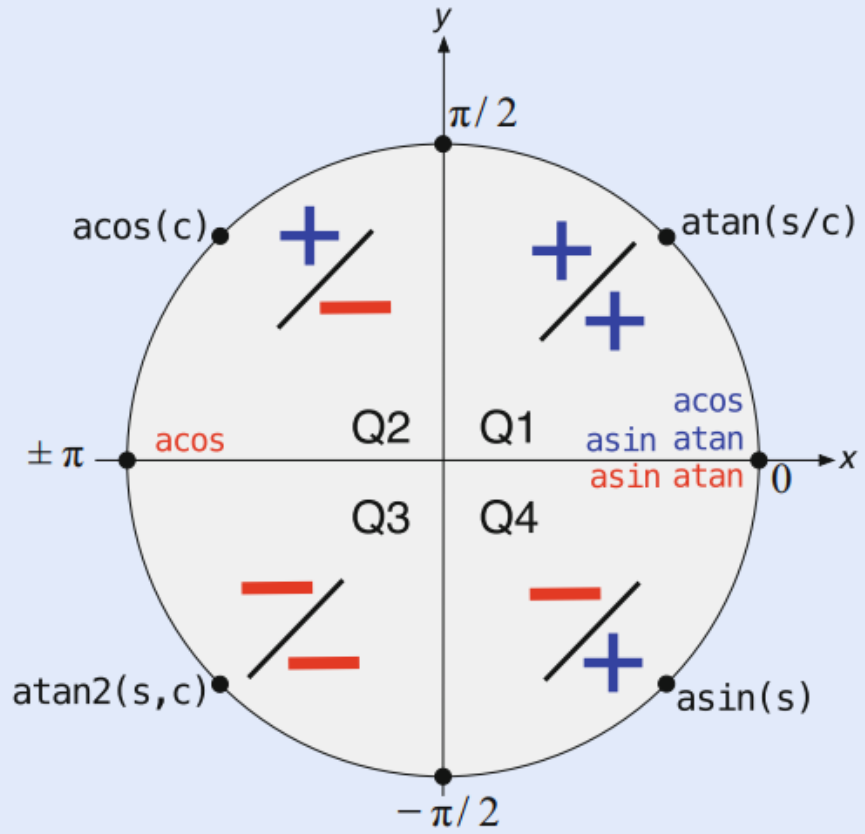
DRAFT

Excurse: `atan` vs `atan2`

In robotics, we frequently need to find θ given an expression for one or more of $\sin\theta$, $\cos\theta$, or $\tan\theta$. The inverse functions are not one-to-one and have ranges of $[-\frac{\pi}{2}, \frac{\pi}{2}]$, $[0, \pi]$ and $[-\frac{\pi}{2}, \frac{\pi}{2}]$ respectively which are all subsets of the full possible range of θ which is $[-\pi, \pi)$.

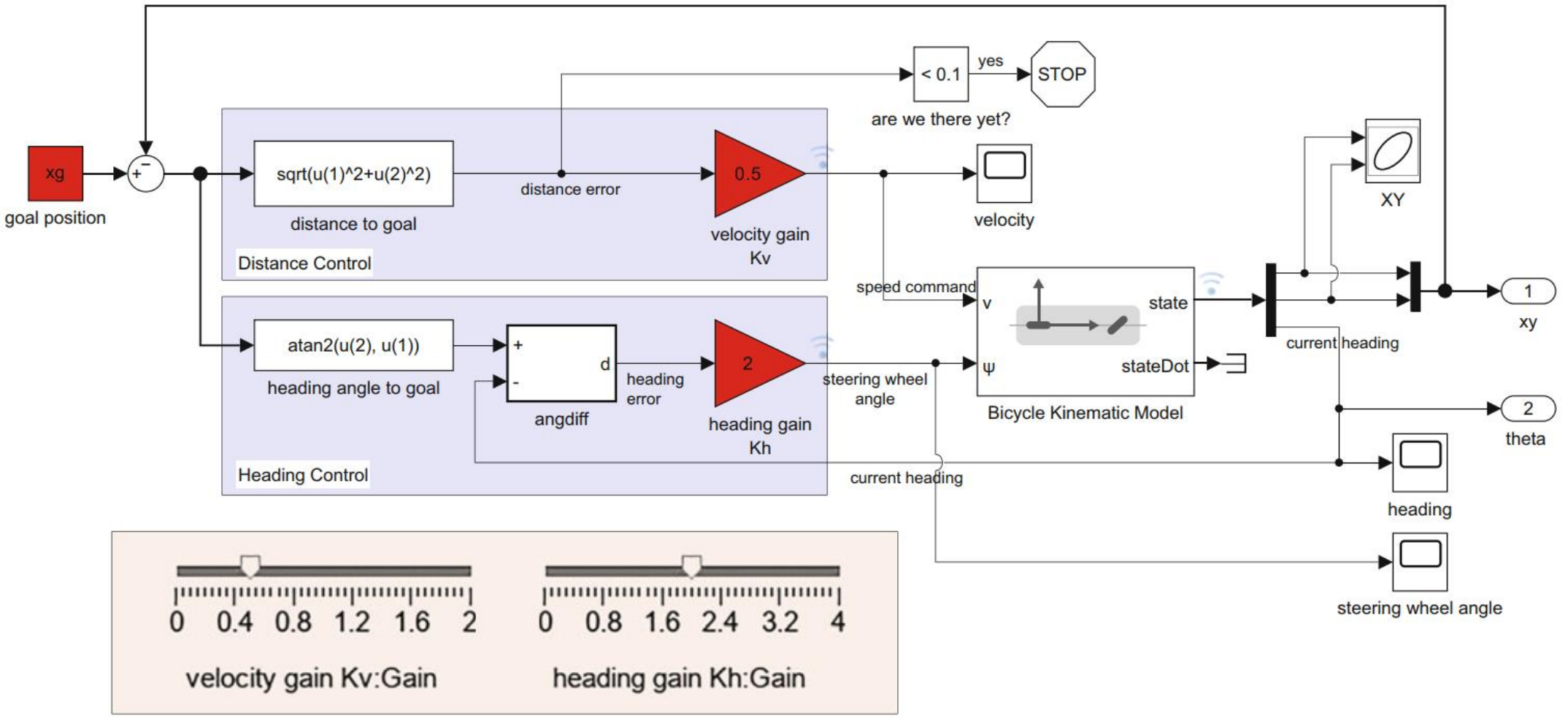
In MATLAB, these inverse functions are implemented by `asin`, `acos`, `atan`, and `atan2`. The figure shows the trigonometric quadrants associated with the sign of the argument to the inverse functions: blue for positive and red for negative.

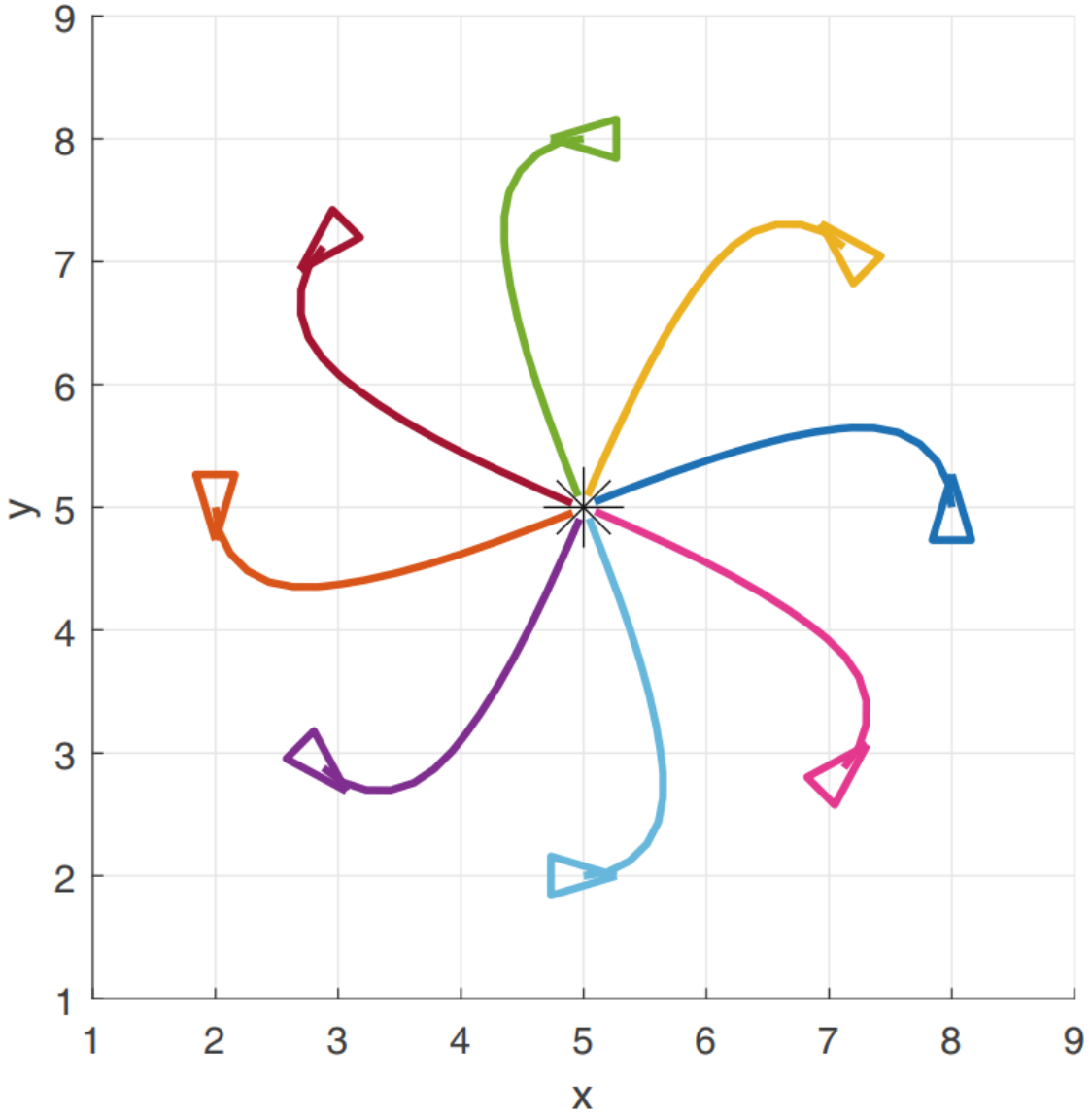
Consider the example for θ in quadrant 3 (Q3), where $\sin\theta = s = \frac{-1}{\sqrt{2}}$ and $\cos\theta = c = \frac{-1}{\sqrt{2}}$. Then `asin(s)` $\rightarrow -\pi/4$, `acos(c)` $\rightarrow 3\pi/4$ and `atan(s/c)` $\rightarrow \pi/4$, three valid but quite different results. For the `atan(s/c)` case, the quotient of the two negative numbers is positive, placing the angle in quadrant 1. In contrast, the `atan2` function is passed the numerator and denominator separately and takes the individual signs into account to determine the angle in any quadrant. In this case, `atan2(s, c)` $\rightarrow -3\pi/4$, which is in quadrant 3.



- A Simulink model

```
>> sl_drivepoint
>> xg = [5 5];
>> x0 = [8 5 pi/2];
>> r = sim("sl_drivepoint");
```

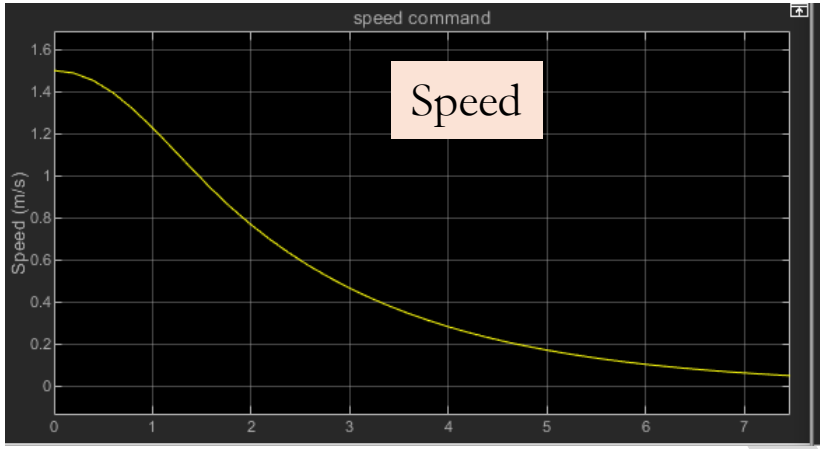




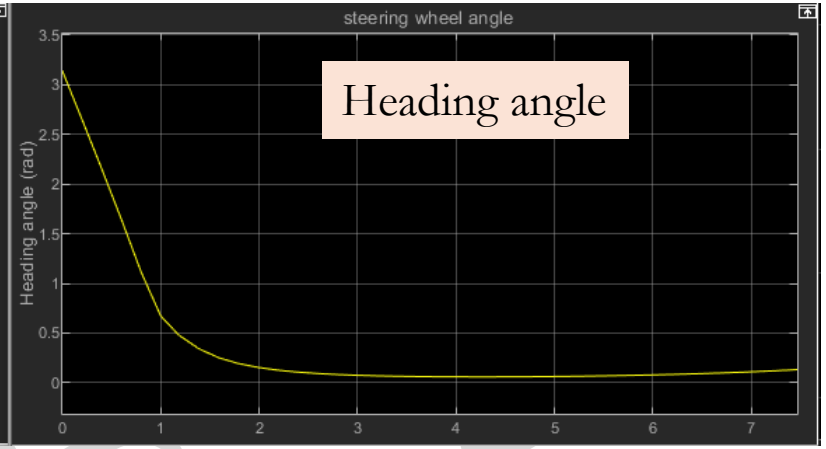
Vehicle motion to a point from different initial configurations.

Speed and heading angle for different K_v and K_h

$K_v = 0.5, K_h = 2$

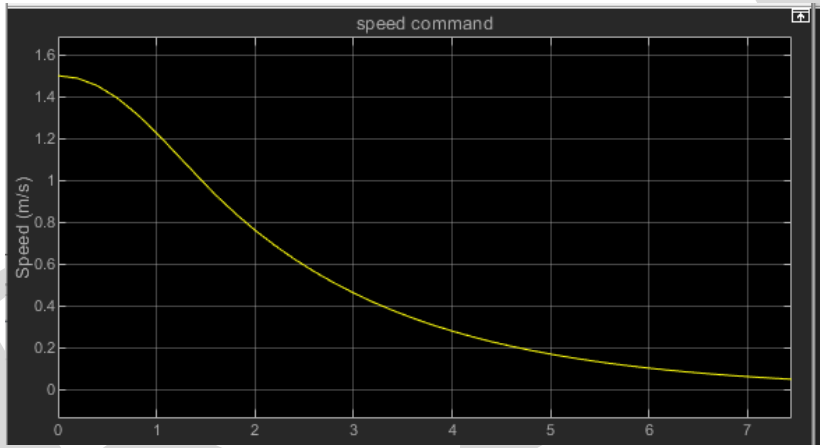


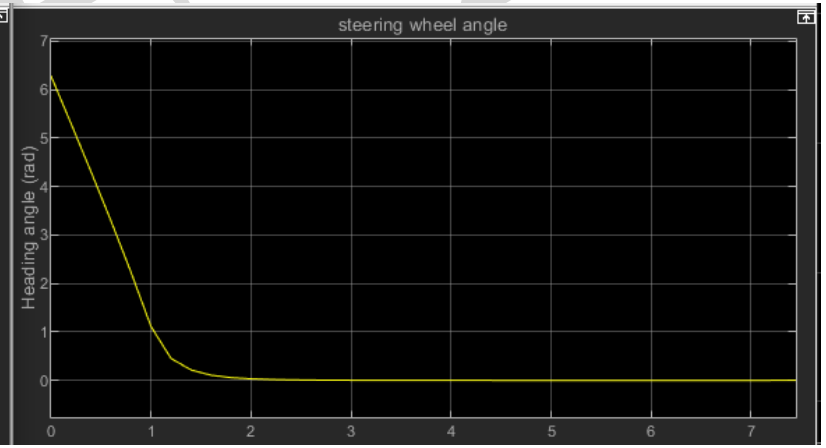
Speed



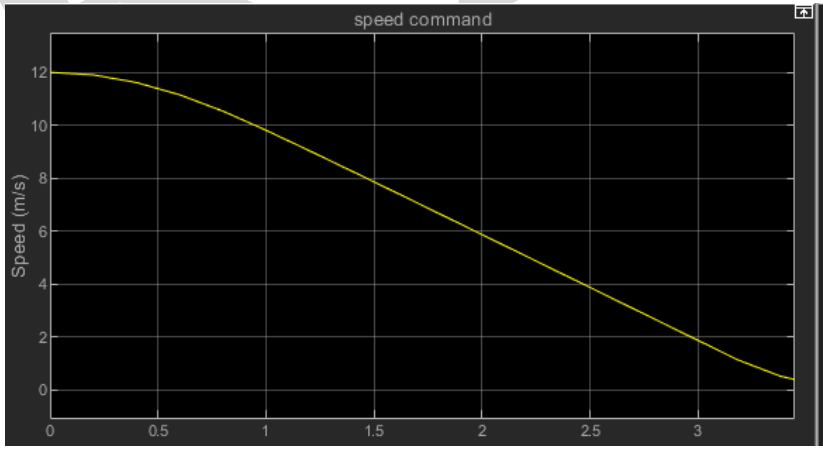
Heading angle

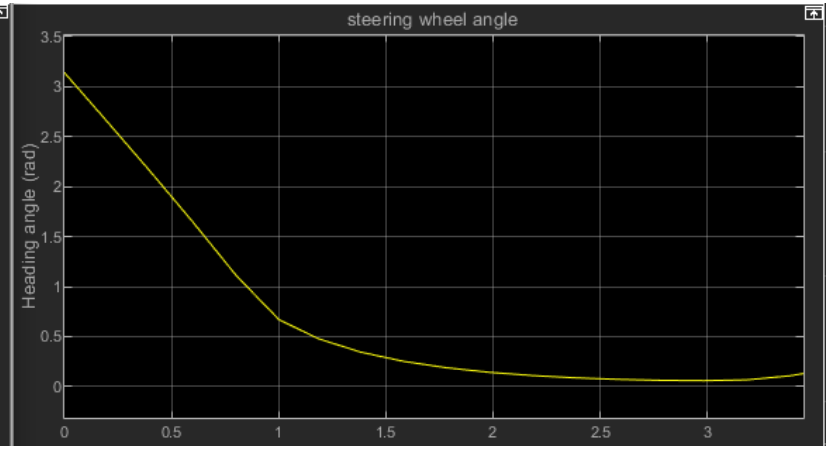
$K_v = 0.5, K_h = 4$





$K_v = 4, K_h = 2$





3.1.4 Driving Along a Line

- Another useful task for a mobile robot is to follow a line on the xy-plane defined by $ax + by + c = 0$.
- We achieve this using two steering controllers.
- The first controller: $\psi_d = K_d d, \quad K_d > 0$

turns the robot toward the line to minimize d, the robot's normal distance from the line

$$d = \frac{(a, b, c) \cdot (x, y, 1)}{\sqrt{a^2 + b^2}}$$

where d > 0 if the robot is to the right of the line.

DRAFT

- The second controller adjusts the heading angle, or orientation, of the vehicle to be parallel to the line

$$\theta^* = \tan^{-1}\frac{a}{-b}$$

using the proportional controller

$$\psi_h = K_h(\theta^* \ominus \theta), \quad K_h > 0$$

- The combined control law

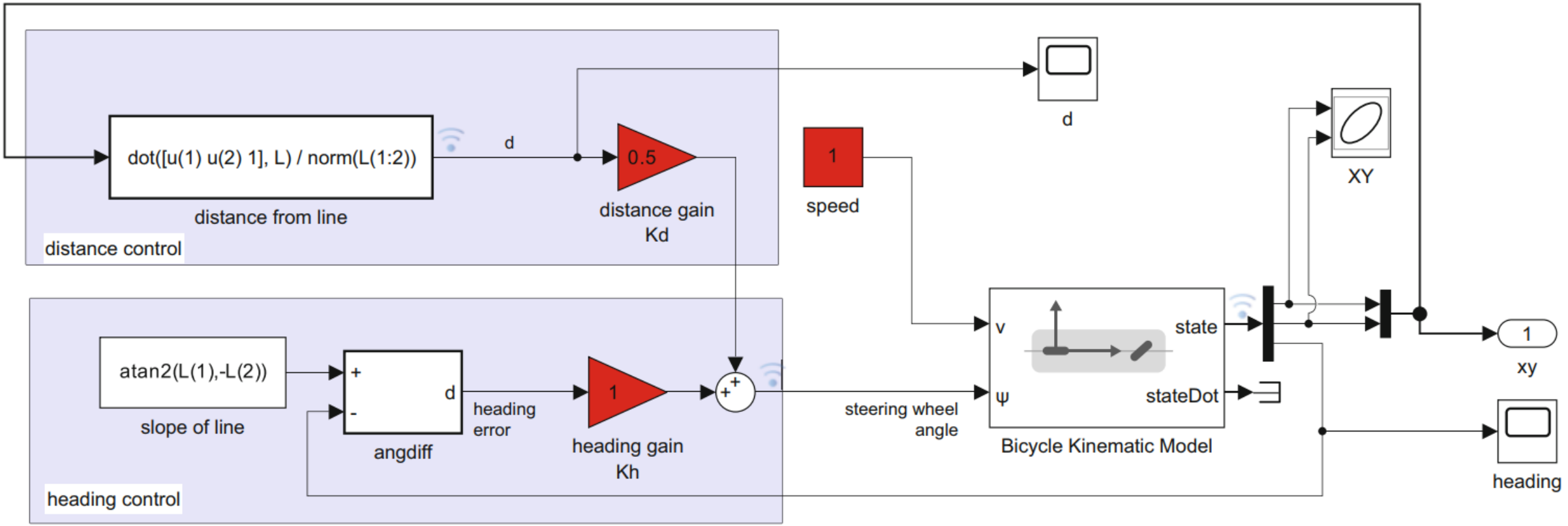
$$\psi = \psi_d + \psi_h = K_d d + K_h(\theta^* \ominus \theta)$$

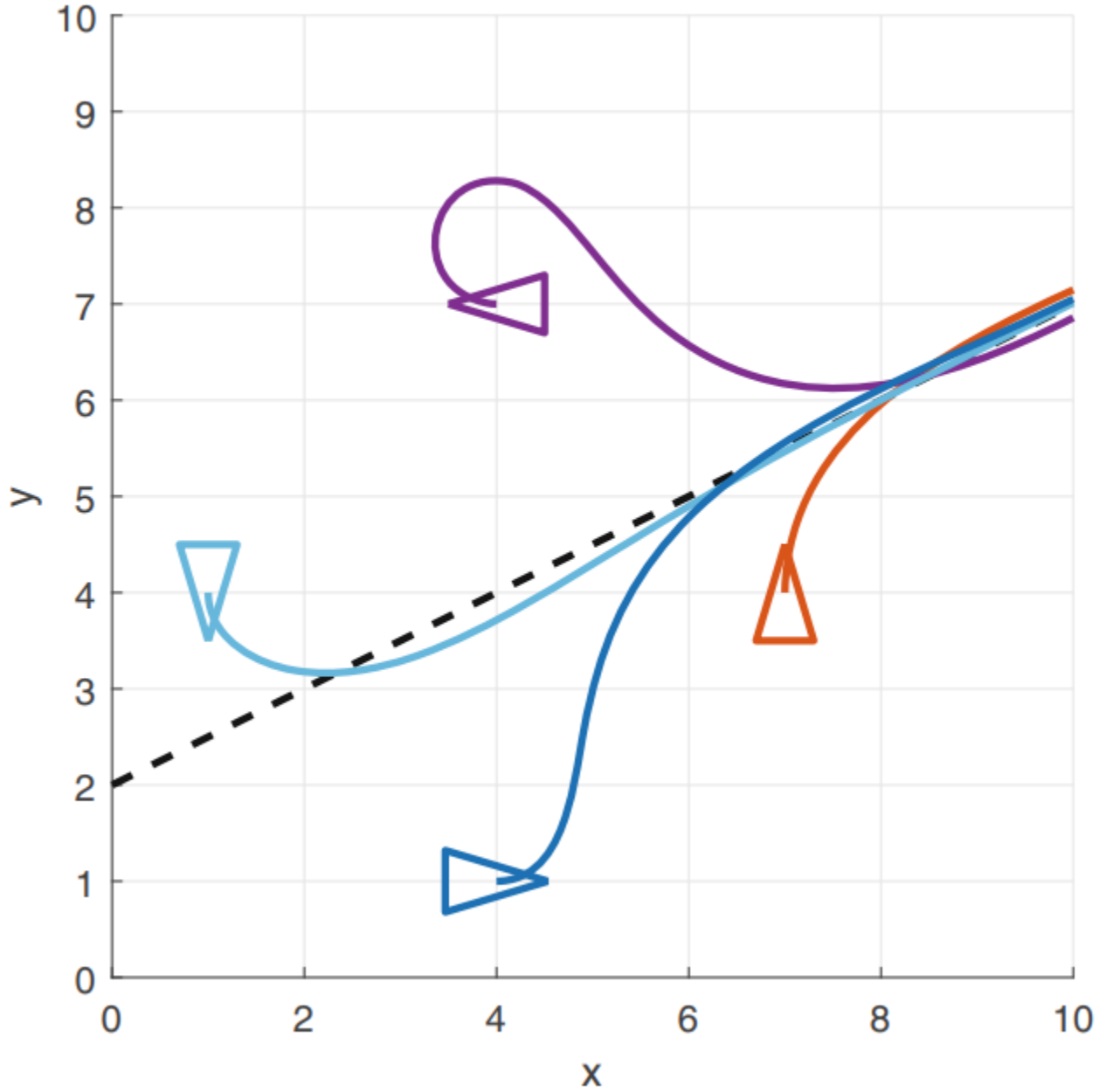
turns the steering wheel to drive the robot toward the line and move along it.

DRAFT

- The Simulink model

```
>> sl_driveline
>> L = [1 -2 4];
>> x0 = [8 5 pi/2];
>> r = sim("sl_driveline");
```





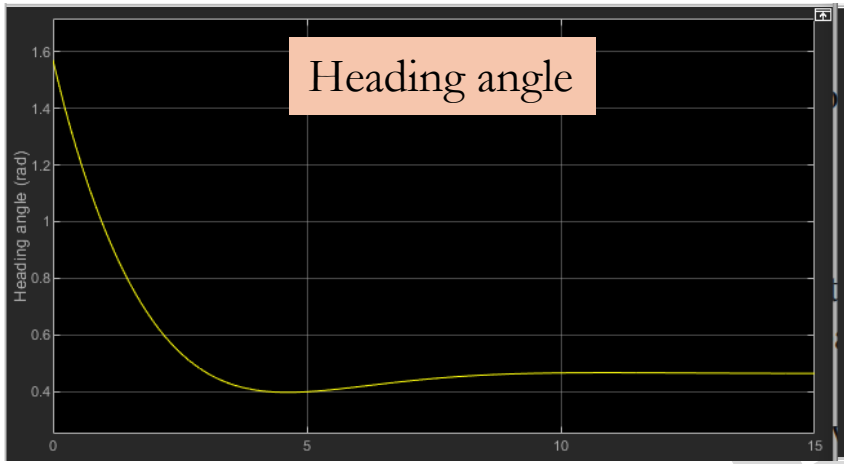
Vehicle motion along a line from different initial configurations.

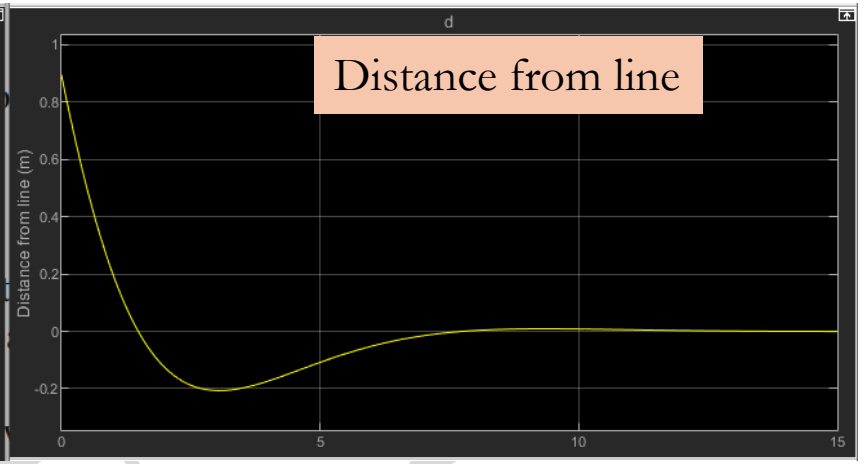
Heading angle and distance from line for different K_d and K_h

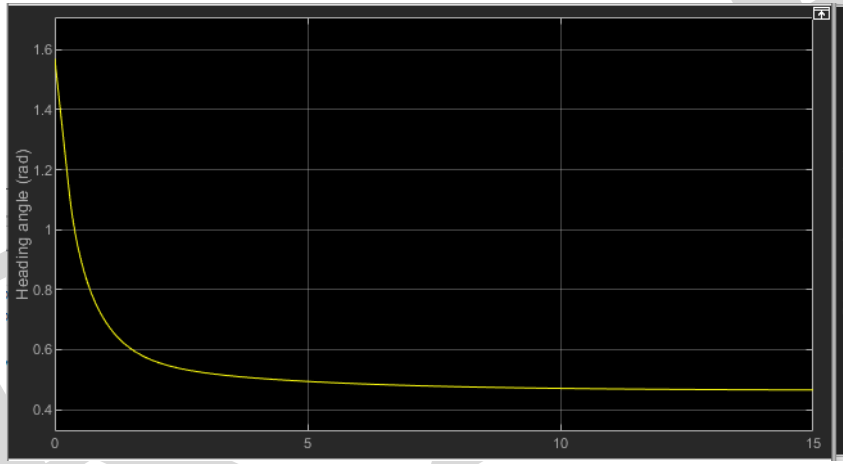
$K_d = 0.5, K_h = 1$

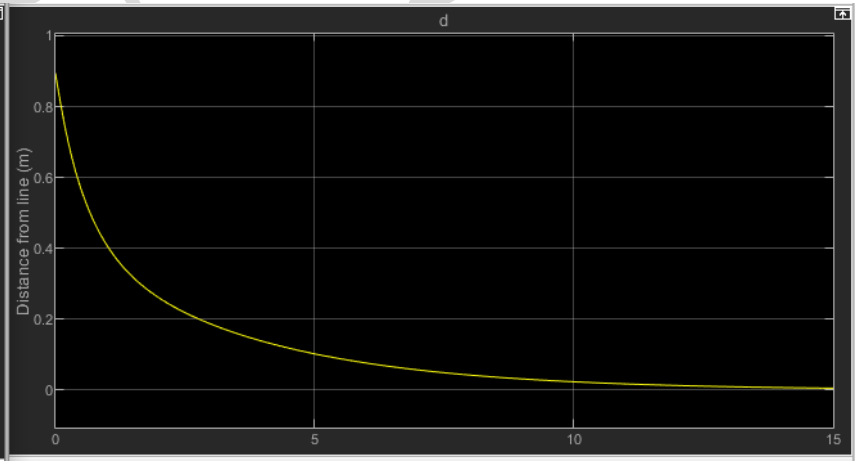
$K_d = 0.5, K_h = 2$

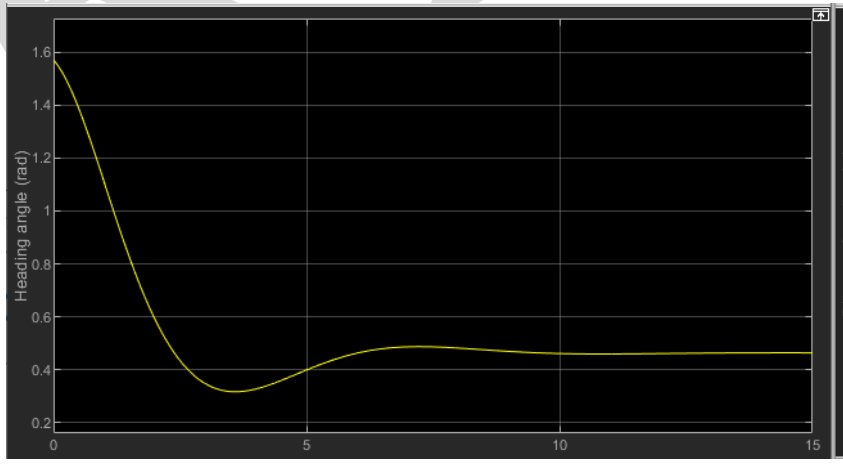
$K_d = 1, K_h = 1$

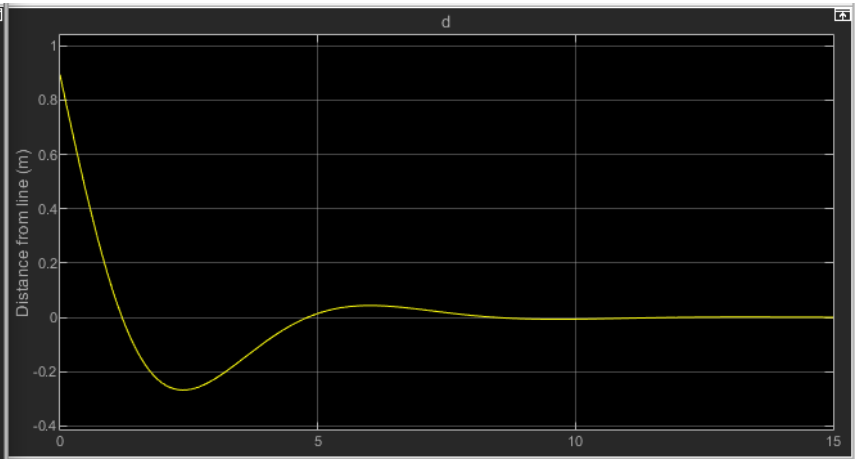












3.1.5 Driving Along a Path

- Instead of following a straight line, we might wish to follow an arbitrary path on the xy-plane. The path might come from one of the path planners.
- A simple and effective algorithm for path following is pure pursuit in which we move at constant velocity and chase a point that is a constant distance ahead on the path.
- The path is defined by a set of N points p_i, $i = 1, \ldots, N$ and the robot's current position is p_r.

DRAFT

- The first step is to find the index of the point on the path that is closest to the robot

$$j = \arg\min_{i=1}^{N} \| \boldsymbol{p}_r - \boldsymbol{p}_i \|$$

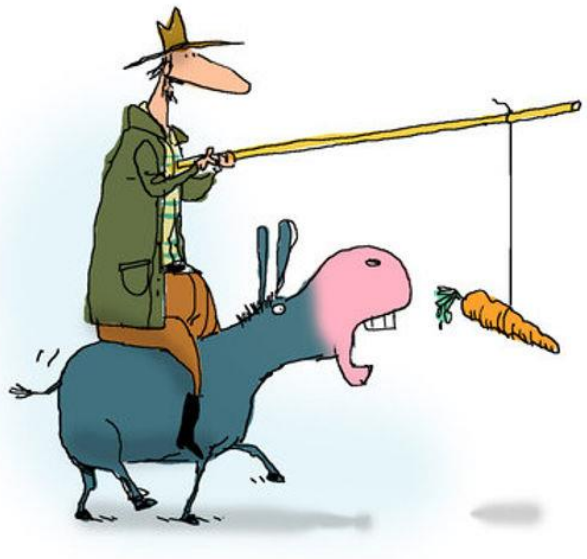
- Next, we find the first point along the path that is at least a distance d ahead of that point

$$k = \arg\min_{i=j+1}^{N} | \| \boldsymbol{p}_i - \boldsymbol{p}_j \| - d |$$

and then the robot drives toward the point p_k.

- This problem is now the same as the one

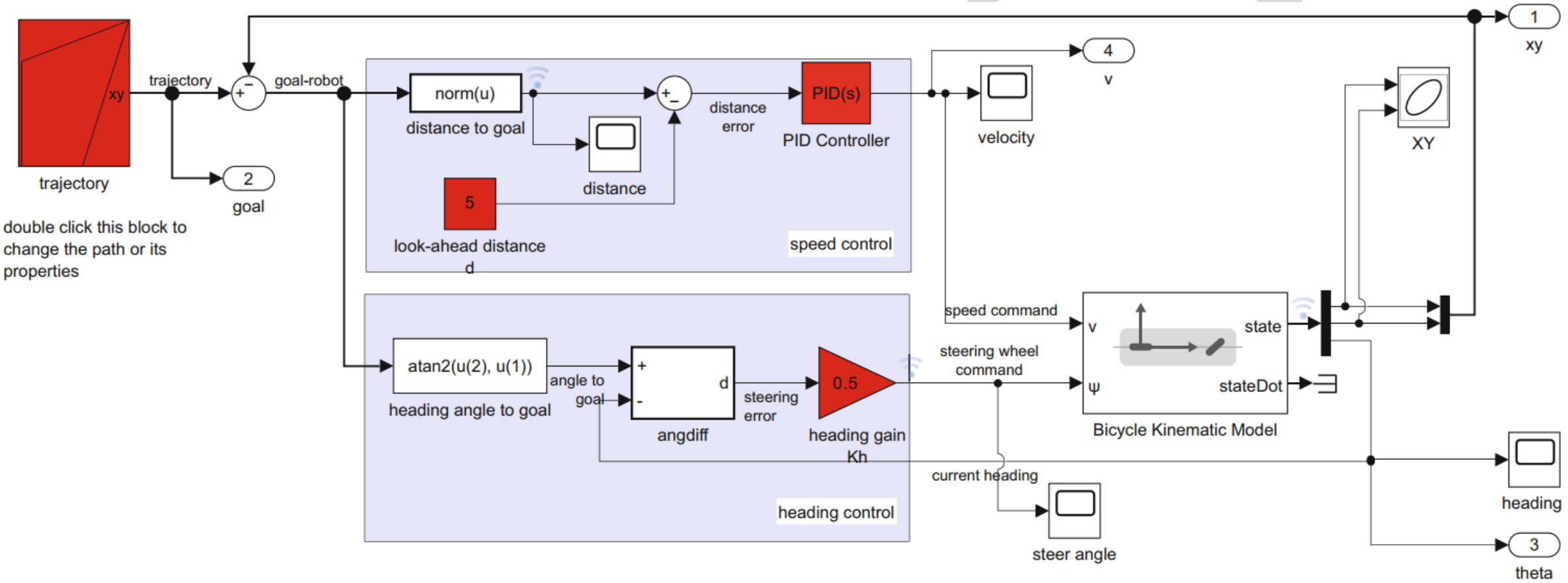
we tackled in Sect. 3.1.3, moving to a point, except that this time the point is moving.

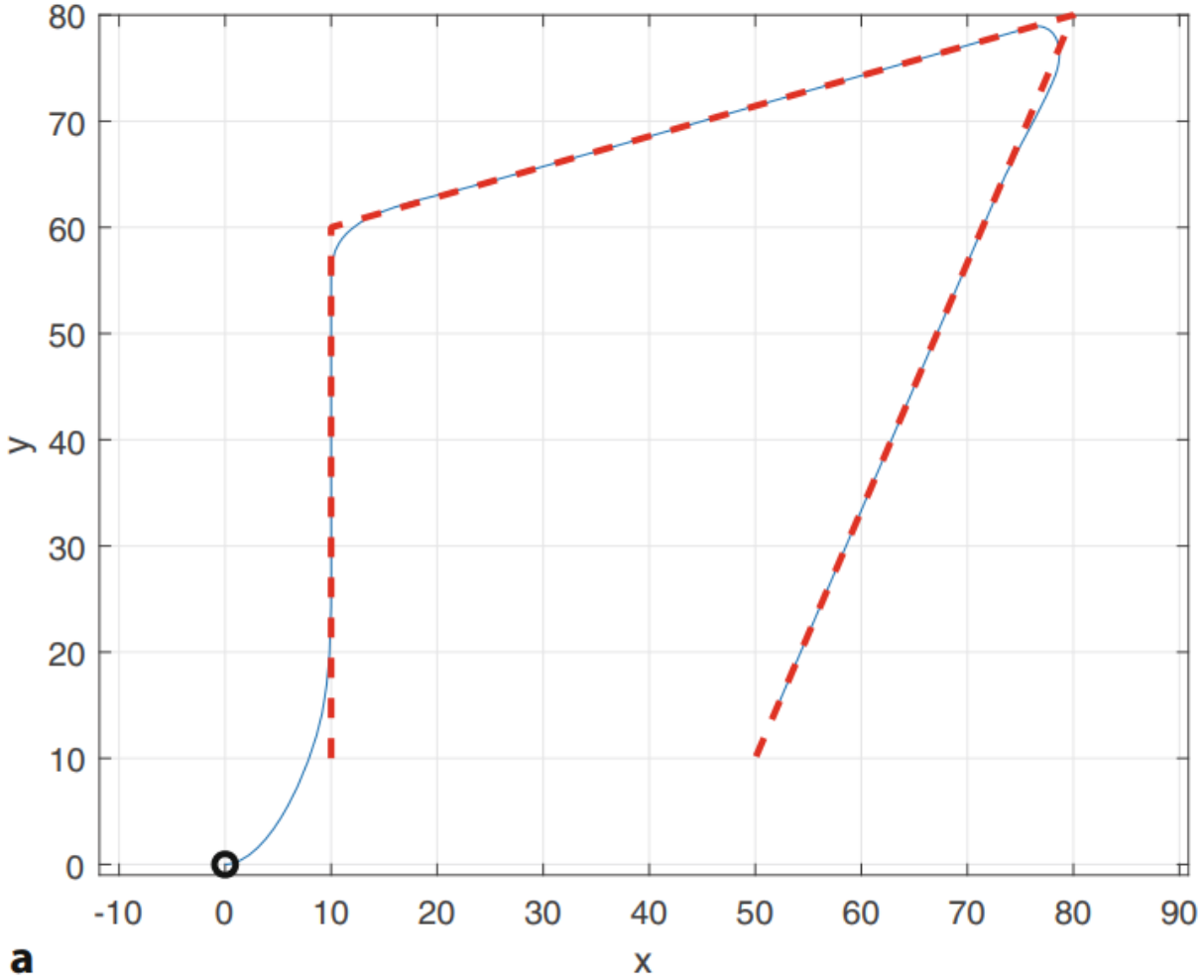


DRAFT

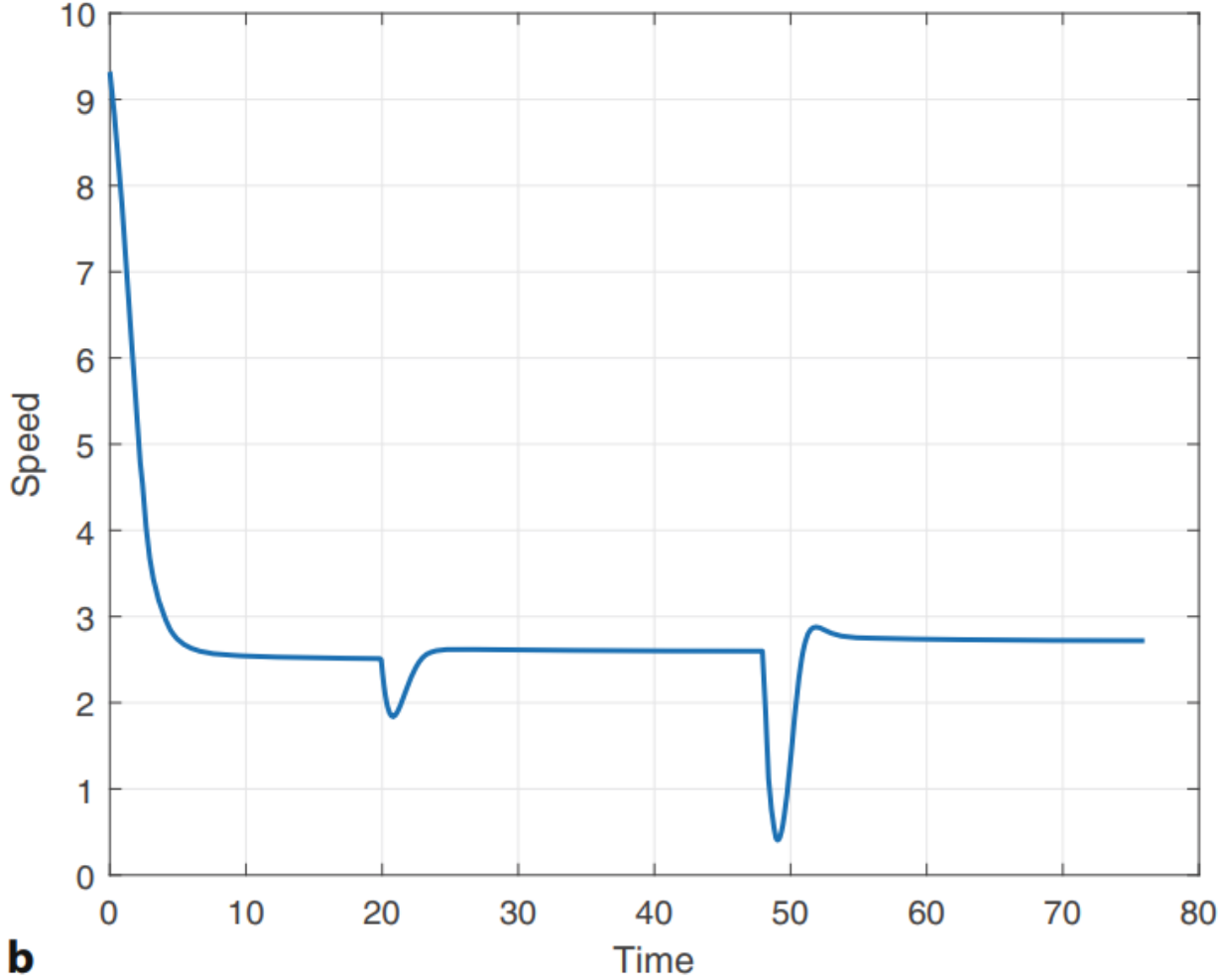
- The Simulink model

```
>> sl_pursuit
>> r = sim("sl_pursuit");
```




80
70
60
50
40
30
20
10
0
y
-10
0
10
20
30
40
50
60
70
80
90
x

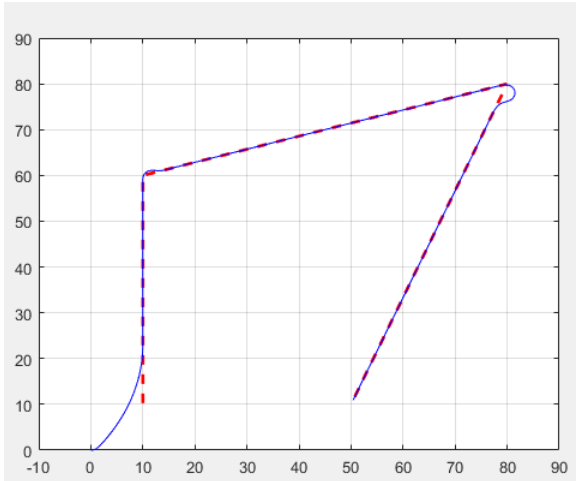
a

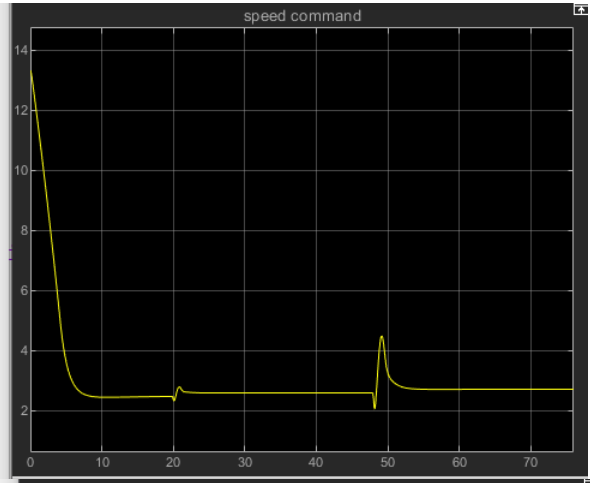

10
9
8
7
6
5
4
3
2
1
0
Speed
0
10
20
30
40
50
60
70
80
Time

b

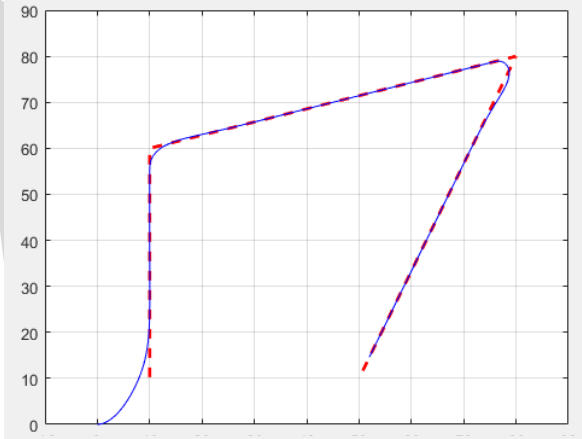
Trajectory and speed for different look-ahead distance

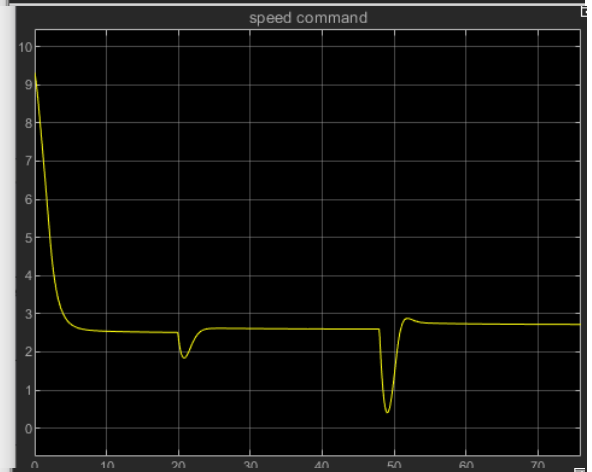
$d = 1$



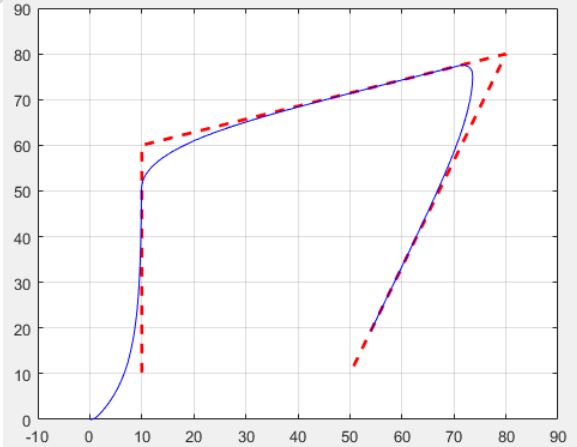


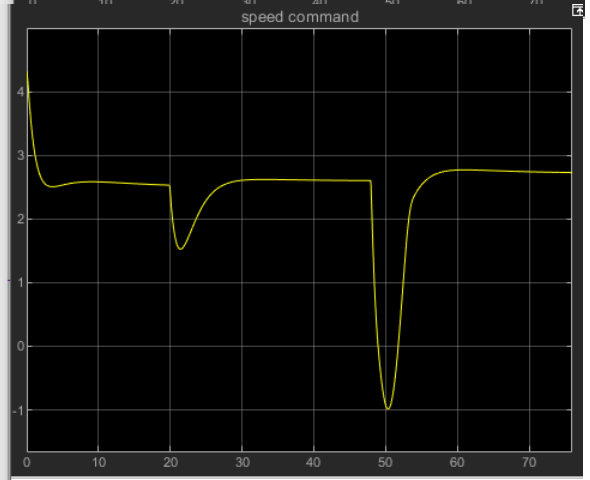
$d = 5$





$d = 10$





DRAFT

3.1.6 Driving to a Configuration

- We discuss is driving to a specific configuration (x^*, y^*, θ^*).
- For tasks like parking, the orientation of the vehicle is important.
- To control the final orientation. we first rewrite the motion model in the matrix form

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix}$$

where the inputs to the vehicle model are the speed v and the turning rate ω which can be achieved by choosing the steered-wheel angle as

$$\psi = \tan^{-1} \frac{\omega L}{v}$$

DRAFT

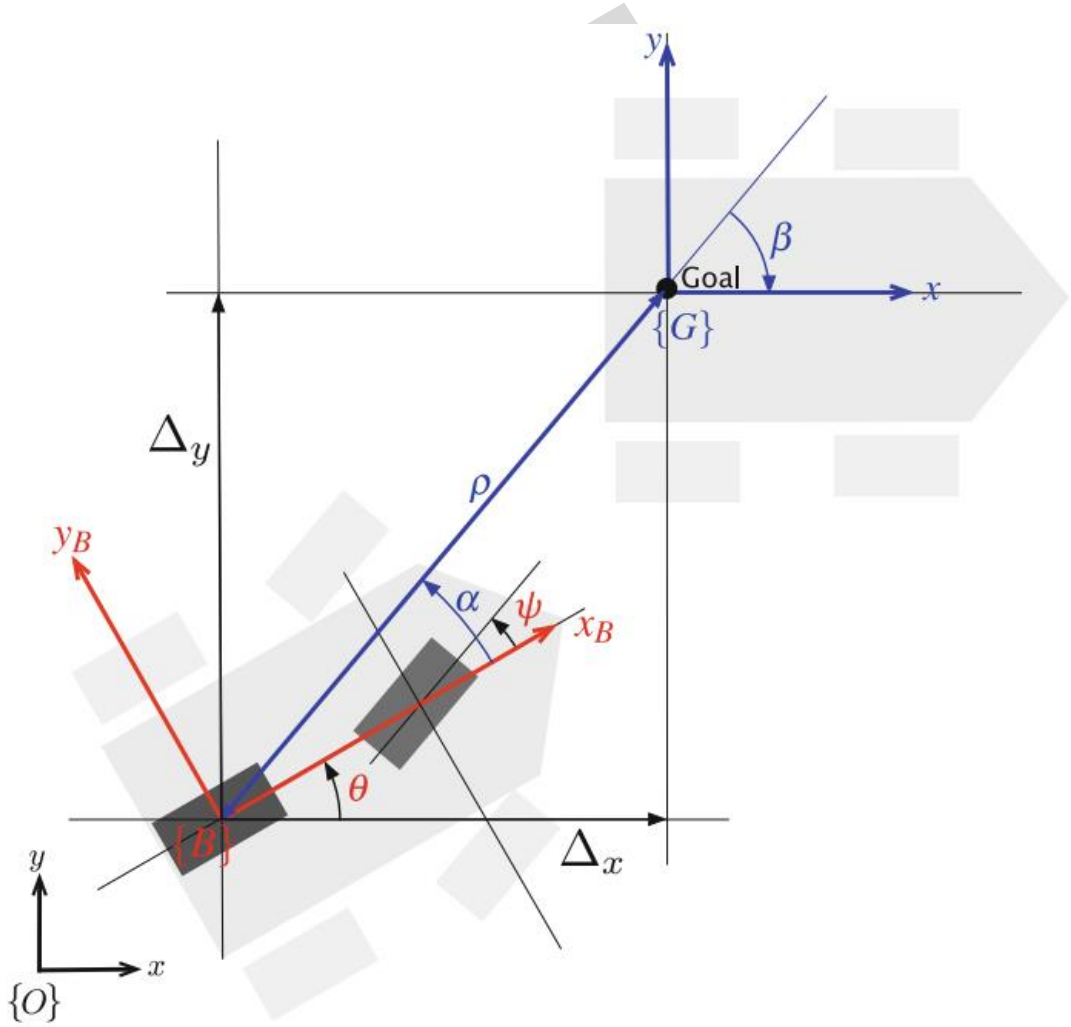
- We then transform the equations into polar coordinate

$$\rho = \sqrt{\Delta_x^2 + \Delta_y^2}$$

$$\alpha = \tan^{-1}\frac{\Delta_y}{\Delta_x} - \theta$$

$$\beta = -\theta - \alpha$$

- Assume that the goal frame {G} is in front of the vehicle.



Polar coordinate notation for the bicycle model vehicle moving toward a goal configuration: ρ is the distance to the goal, β is the angle of the goal vector with respect to the world frame, and α is the angle of the goal vector with respect to the vehicle frame.

DRAFT

- The linear control law

$$v = K_\rho \rho$$
$$\omega = K_\alpha \alpha + K_\beta \beta$$

drives the robot to a unique equilibrium at $(\rho, \alpha, \beta) = (0,0,0)$.

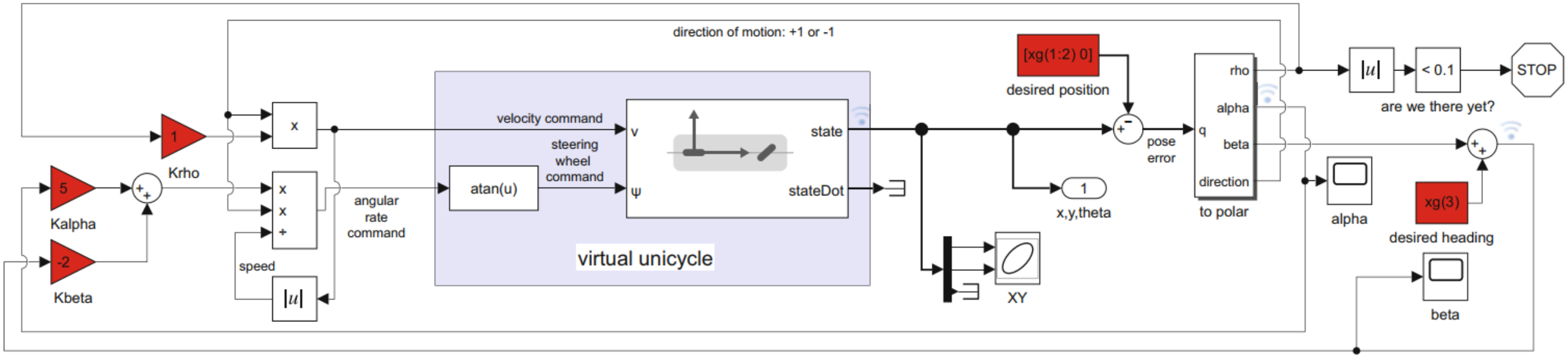
- The intuition behind this controller is that the terms $K_\rho \rho$ and $K_\alpha \alpha$ drive the robot along a line toward {G} while the term $K_\beta \beta$ rotates the line so that $\beta \to 0$.
- To implement this in practice, the distance and bearing angle to the goal (ρ, α) could be measured by a camera or lidar, and the angle β could be derived from α and vehicle heading θ as measured by a compass.
- To move the robot to an arbitrary configuration (x^*, y^*, θ^*), we perform a change of coordinates

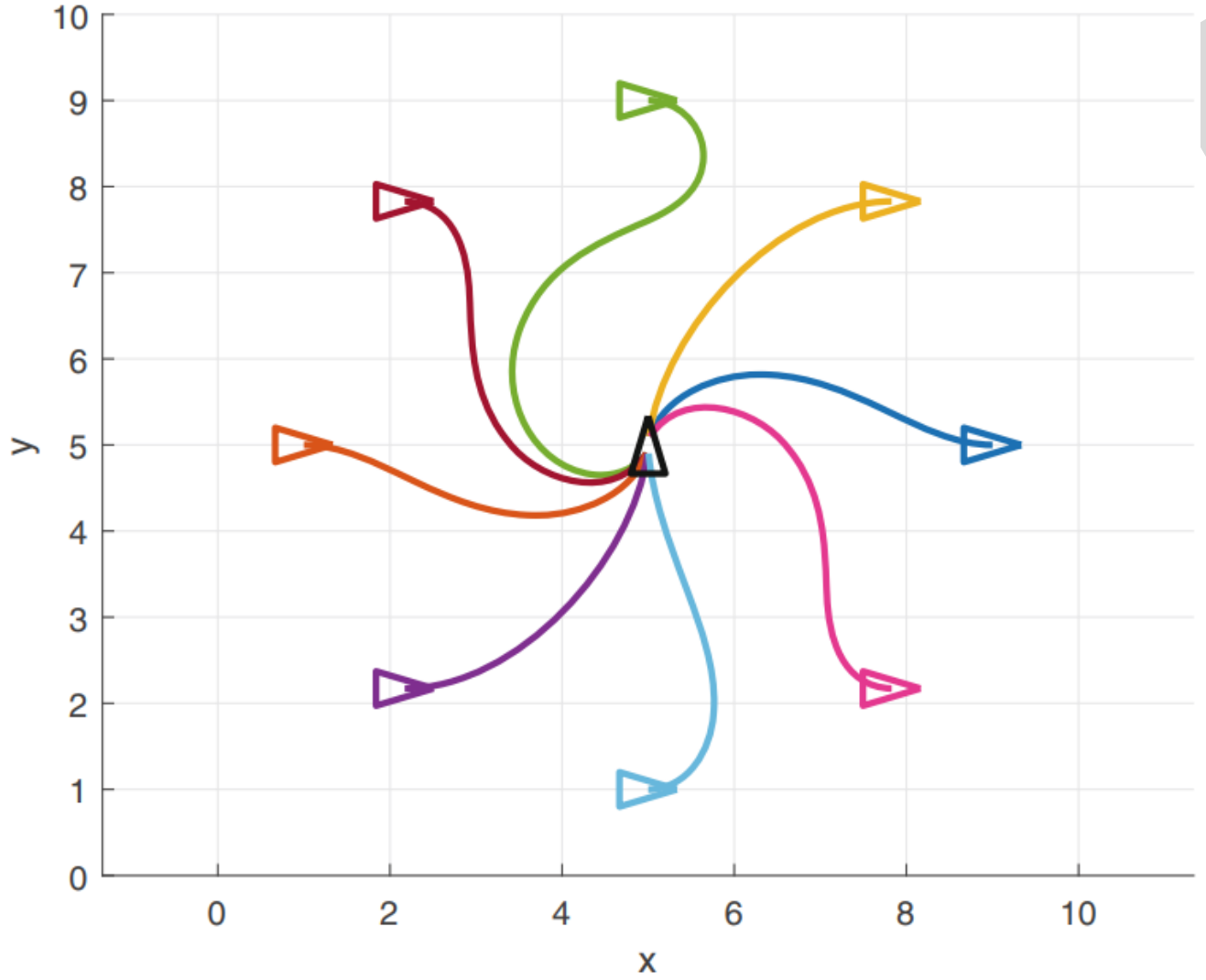
$$x' = x - x^*, \quad y' = y - y^*, \quad \theta' = \theta, \quad \beta = \beta' + \theta^*$$

DRAFT

- The Simulink model

```
>> sl_drivepose
>> xg = [5 5 pi/2];
>> x0 = [9 5 0];
>> r = sim("sl_drivepose");
>> q = r.find("y");
>> plot(q(:,1),q(:,2));
```





Vehicle motion to a configuration from different initial configurations.